Automated Deployment of Web Application Using AWS CodePipeline

Project Overview

Project Title

Automated Deployment of Web Application Using AWS CodePipeline

Team Size

5 Members

Project Duration

2 Weeks

Project Objective

The objective of this project is to demonstrate how a web application can be automatically built and deployed using AWS services. The project will implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline that automatically detects code changes, builds the application, and deploys it to a hosting environment without manual intervention.

Problem Statement

Traditional software deployment requires developers to manually upload files and update applications whenever changes are made. This process is time-consuming, prone to human error, and difficult to manage in larger projects.

The proposed solution uses AWS CodePipeline to automate the deployment process. Whenever developers push code changes to a GitHub repository, AWS services automatically build and deploy the updated application.

Project Goals

Primary Goals

1. Understand CI/CD concepts.
2. Learn AWS cloud services.

3. Build an automated deployment pipeline.
4. Deploy a web application using AWS.
5. Reduce manual deployment efforts.
6. Demonstrate DevOps practices.

Learning Outcomes

By the end of the project, team members will understand:

- Git and GitHub
- AWS Cloud Basics
- AWS IAM
- AWS CodePipeline
- AWS CodeBuild
- Amazon S3 Hosting
- Continuous Integration
- Continuous Deployment
- Cloud-Based Automation

Project Architecture

Code Change ↓ GitHub Repository ↓ AWS CodePipeline ↓ AWS CodeBuild ↓ Amazon S3 ↓ Live Website

Components Used

1. GitHub
2. Source code repository
3. Stores project files
4. AWS CodePipeline
5. Automates workflow
6. Coordinates deployment stages
7. AWS CodeBuild
8. Builds application
9. Generates deployment artifacts
10. Amazon S3

11. Hosts website

12. Serves application files

Project Scope

Included

- Simple responsive website
- Source code management using GitHub
- Automated deployment pipeline
- Build automation
- Website hosting on AWS
- Deployment testing

Excluded

- Complex backend systems
- Databases
- User authentication
- Microservices architecture
- Advanced monitoring

Team Roles and Responsibilities

Member 1 – Project Manager & Documentation Lead

Responsibilities:

- Coordinate team activities
- Track project progress
- Prepare documentation
- Create architecture diagrams
- Prepare presentation slides
- Final project submission

Deliverables:

- Project report
- PPT presentation
- Architecture diagrams

Member 2 – GitHub & Source Control Lead

Responsibilities:

- Create GitHub repository
- Manage project structure
- Handle commits and branches
- Ensure code synchronization

Deliverables:

- GitHub repository
- Version control documentation

Member 3 – Amazon S3 Deployment Lead

Responsibilities:

- Create S3 bucket
- Configure static website hosting
- Set bucket permissions
- Verify deployment results

Deliverables:

- Hosted website
- S3 configuration screenshots

Member 4 – AWS CodeBuild Lead

Responsibilities:

- Create build project
- Configure build process
- Write buildspec.yml
- Test build stages

Deliverables:

- Build configuration
- Successful build logs

Member 5 – AWS CodePipeline Lead

Responsibilities:

- Create deployment pipeline
- Connect GitHub to AWS
- Configure deployment stages
- Test automation workflow

Deliverables:

- Working pipeline
- Pipeline execution screenshots

Project Deliverables

1. GitHub Repository
2. Source Code
3. AWS Infrastructure Setup
4. Working CI/CD Pipeline
5. Hosted Website
6. Documentation
7. Presentation Slides
8. Final Demonstration

Two-Week Execution Plan

Week 1

Day 1

- Understand project requirements
- Learn CI/CD basics
- Study AWS services

Day 2

- Create GitHub repository
- Design website structure

Day 3

- Develop website pages

- Create HTML and CSS files

Day 4

- Complete website development
- Test website locally

Day 5

- Configure Amazon S3
- Deploy website manually

Day 6

- Configure AWS CodeBuild
- Create build project

Day 7

- Configure AWS CodePipeline
- Connect GitHub and S3

Week 2

Day 8

- Execute first automated deployment

Day 9

- Fix issues and permissions

Day 10

- Improve website design

Day 11

- Test multiple deployments

Day 12

- Capture screenshots
- Collect deployment logs

Day 13

- Prepare documentation

- Create presentation slides

Day 14

- Final testing
- Practice project demonstration

Implementation Steps

Step 1

Create GitHub repository.

Step 2

Develop web application.

Step 3

Create Amazon S3 bucket.

Step 4

Enable static website hosting.

Step 5

Create AWS CodeBuild project.

Step 6

Create buildspec.yml file.

Step 7

Create AWS CodePipeline.

Step 8

Connect GitHub repository.

Step 9

Configure deployment stage.

Step 10

Test automatic deployment.

Success Criteria

The project will be considered successful if:

1. Website is hosted successfully.
2. GitHub repository is connected.
3. CodePipeline executes automatically.
4. CodeBuild completes successfully.
5. Deployment occurs without manual intervention.
6. Website updates after code changes.

Demonstration Plan

Initial State

Website displays: "Version 1"

Action

1. Modify website content.
2. Change text to "Version 2".
3. Commit changes.
4. Push code to GitHub.

Automated Process

1. GitHub detects change.
2. CodePipeline starts.
3. CodeBuild executes.
4. Deployment completes.

Final Result

Website automatically updates and displays: "Version 2"

This demonstrates a fully functional CI/CD pipeline.

Risks and Mitigation

Risk 1

IAM Permission Errors

Mitigation: Review IAM roles and policies carefully.

Risk 2

Pipeline Configuration Errors

Mitigation: Test each stage individually.

Risk 3

GitHub Integration Issues

Mitigation: Verify repository access permissions.

Risk 4

Deployment Failures

Mitigation: Review build logs and deployment logs.

Future Enhancements

1. Add CloudFront CDN.
2. Add Deployment Notifications.
3. Add Manual Approval Stage.
4. Add Development and Production Environments.
5. Add Automated Testing.
6. Deploy Dynamic Applications.

Conclusion

This project demonstrates the implementation of a Continuous Integration and Continuous Deployment (CI/CD) pipeline using AWS services. The solution automates the process of building and deploying a web application whenever code changes are made. Through this project, team members gain practical experience with cloud computing, DevOps practices, and AWS deployment services while creating a reliable and scalable deployment workflow.